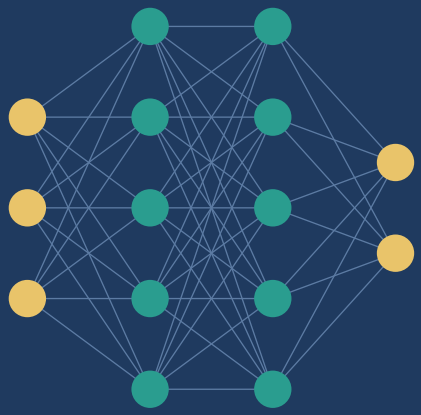
МОДУЛЬ 3 · ВАЙБКОДИНГ · УРОК 9

Первый проект

От идеи до работающего инструмента: ежемесячная презентация



О чём этот урок

В уроке 8 мы разобрались, что такое вайбкодинг. Теперь пройдём полный путь одного проекта: от идеи до работающего инструмента. Задача — настоящая, из практики финансовой службы: **ежемесячная презентация для руководства**. 11 слайдов в фирменном стиле, цифры из управленческой отчётности, комментарии подразделений. Сейчас её собирают вручную, полдня каждый месяц.

Главное

После урока вы сможете:

- написать спецификацию — карту «слайд → источник → правило»;
- отделить оформление (шаблон) от данных (скрипт);
- собирать программу с ИИ по шагам, проверяя каждый шаг;
- принять результат: сверить с ручной версией и проверить на «сломанных» данных.

Время: ~50 минут чтения + 40 минут практики. Весь проект из урока лежит в репозитории курса, в папке examples/monthly_pptx: спецификация, шаблон, выдуманные данные, скрипт и готовый результат.

1. Что получится в итоге

Сначала договоримся о главном, чтобы не потеряться в деталях. **Результат урока — не ответ ИИ, а программа**: файл со скриптом на Python (build_presentation.py) и шаблон презентации (template.pptx). ИИ в проекте работает в двух разных ролях:

- **Один раз — помогает написать скрипт.** Вы ведёте с моделью разговор из нескольких промптов (вайбкодинг, урок 8). Итог разговора — файл со скриптом, который вы проверили на прошлом месяце.
- **Каждый месяц — только запускает готовый скрипт.** Новые файлы Excel загружают в чат с песочницей для кода или отдают ИИ-агенту, и на выходе получают pptx. Код при этом не пишется заново и не меняется.

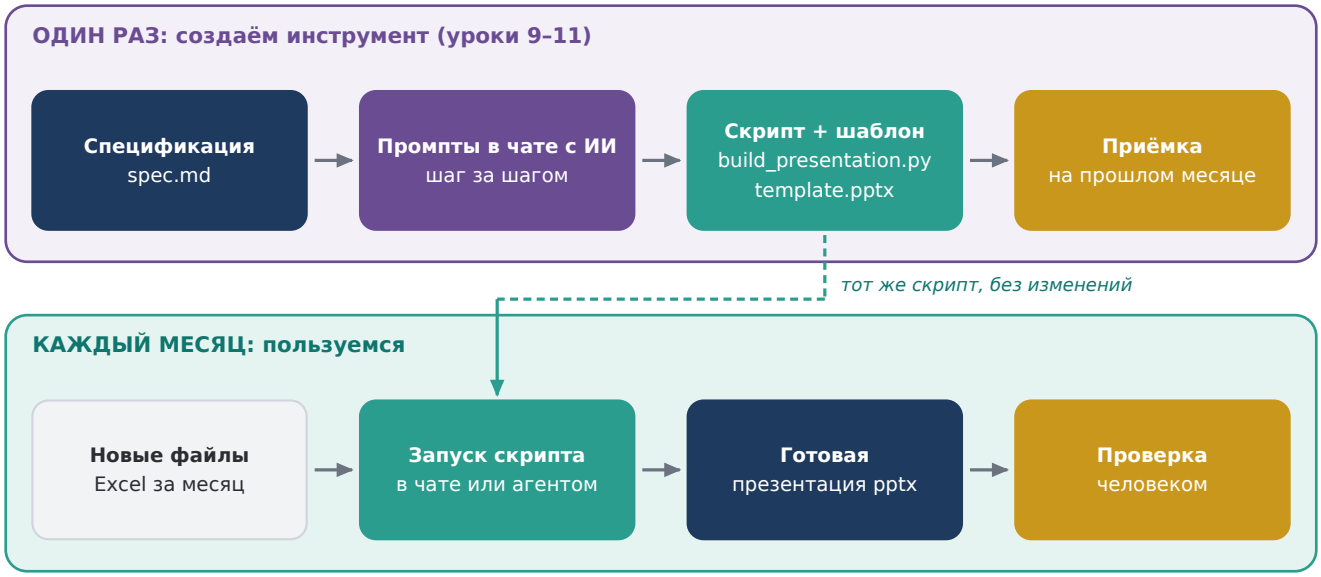


Рис. 1. Две фазы проекта: создать инструмент один раз — пользоваться каждый месяц

Главное

Цифры на слайды переносит **программа**, а не модель «из головы». Модель помогает эту программу написать — один раз. Каждый месяц потом работает уже проверенный код (урок 8, рис. 1). Где именно его запускать — в отдельном чате или внутри ИИ-агента, — разберём в разделе 6 и в уроках 14–16.

Идея и границы

Любой проект начинается не с кода, а с ответа на три вопроса: **что** автоматизируем, **что нет** и **как поймём, что получилось**.

Вопрос	Ответ для нашей презентации
Что автоматизируем	Цифры, таблицы, графики, вставку готовых комментариев, служебный слайд с контрольными суммами
Что НЕ автоматизируем	Оценки и выводы: их пишут люди, скрипт вставляет текст дословно
Как поймём, что получилось	Прогон на прошлом месяце совпал с ручной версией; на «сломанных» данных скрипт останавливается с понятной ошибкой

Окупится ли

Простая прикидка: если ручная сборка занимает 4 часа в месяц, а проект с отладкой — 2–3 рабочих дня, он окупается за полгода. И это без учёта главного: исчезают ошибки переноса цифр, а сборку перестаёт держать «в голове» один человек.

Частое сомнение

«Проще собрать руками, чем всё это автоматизировать»

Для разовой презентации — да, безусловно. Автоматизация окупается там, где одна и та же работа повторяется каждый месяц. И выигрыш не только во времени: копирование цифр вручную — главный источник ошибок в регулярных отчётах.

Что здесь справедливо. первый запуск действительно дороже, чем один ручной вариант, а при каждом изменении структуры отчёта скрипт придётся дорабатывать. Если структура презентации меняется каждый месяц, автоматизировать её рано.

В разговоре с коллегой: *«Давай посчитаем: сколько часов в год уходит на эту презентацию и сколько займёт скрипт.»*

2. Спецификация — главный документ проекта

Спецификация — это описание задачи настолько точное, что по нему можно написать программу, не задавая вопросов. Тест нового сотрудника из урока 4, только строже. И пишет её **владелец отчёта**, а не программист: только вы знаете, откуда берётся каждая цифра и как её округлять.

В спецификации четыре части:

1. **Цель и границы** — см. раздел 1.
2. **Входные данные:** какие файлы, листы, колонки и строки. Строки ищем по подписи («Итого»), а не по номеру — номер сдвинется, как только кто-то вставит строку.
3. **Проверки входа:** что должно быть верно, чтобы вообще собирать презентацию.
4. **Карта «слайд → источник → правило»** — сердце документа. Например, слайд «Долговая нагрузка» берёт долг и EBITDA[1] с листа «Долг» и считает их отношение.



Полная карта на 11 слайдов — в файле spec.md примера проекта

Рис. 2. Фрагмент карты: для каждого слайда — откуда цифры и как их считать

[1] **EBITDA** — Earnings Before Interest, Taxes, Depreciation and Amortization: прибыль до вычета процентов, налогов и амортизации.

Шаг 0. Как получить spec.md

Писать спецификацию с чистого листа не нужно: её удобно составить **вместе с моделью**, в отдельном чате. Вы даёте структуру презентации и исходников, модель задаёт вопросы (приём «задай мне вопросы» из урока 4) и собирает черновик по разделам. Ваша работа — ответить на вопросы и выверить черновик.

```
Я веду ежемесячную презентацию для руководства и хочу автоматизировать
её сборку скриптом на Python. Сначала нужна спецификация — точное
описание задачи, по которому потом напишут скрипт.

Структура презентации: [список слайдов: № | название | что на нём —
таблица, график или текст; или файл презентации прошлого месяца]
Исходники: [файлы, листы, заголовки колонок, подписи строк;
цифры замените выдуманными]

Сначала задай мне до 10 вопросов о том, чего не хватает: откуда
каждая цифра, как её считать и округлять, что делать при ошибках
в данных, откуда берутся тексты.
После моих ответов составь спецификацию в формате Markdown
с разделами: 1) цель и границы; 2) входные данные (таблица: файл |
лист | колонки | строки); 3) проверки входа; 4) карта «слайд →
источник → правило» (таблица); 5) шаблон; 6) приёмка.
Ничего не придумывай: где не знаешь — пиши «УТОЧНИТЬ».
```

Дальше — три действия:

1. Ответьте на вопросы модели. Не знаете ответа — значит, это стоит выяснить ещё до кода.
2. Выверите черновик: каждую строку карты сравните с ручной презентацией, все «УТОЧНИТЬ» закройте.
3. Сохраните текст как файл **spec.md**. Markdown — это обычный текст с простой разметкой (# — заголовок, | — таблица); его открывает Блокнот, а модели хорошо понимают такую разметку. Подойдёт и spec.txt.

Осторожно

Ручная презентация прошлого месяца содержит настоящие цифры. Отдавать её целиком можно только в разрешённый ИИ — например, в локальный ассистент компании. В публичный чат — только структуру: список слайдов и названия колонок, а цифры замените выдуманными (уроки 3 и 6).

Готовый spec.md — первый файл проекта. Дальше он прикладывается к каждому промпту шагов 1–6 и к каждому новому чату. Образец лежит в примере проекта: examples/monthly_pptx/spec.md.

Проверки входа

Исходники ведут люди, и структура иногда меняется — колонку переименовали, строку вставили, итог забыли пересчитать. Скрипт без проверок молча соберёт презентацию с пустым или неверным слайдом. Поэтому первые строки спецификации — что проверить **до** сборки:

- все файлы, листы и заголовки колонок на месте;
- в числовых ячейках числа, а не текст;
- «Итого» равно сумме направлений;
- одна и та же цифра на разных листах совпадает (факт в «Сводке» = последний месяц в «Динамике»);
- у каждого слайда есть комментарий, если он нужен.

Частое сомнение

«Скрипт сломается, как только поменяют таблицу»

Сломается — и это хорошо, если он сломается громко. Плохо, когда программа молча собирает неверную презентацию. Проверки входа превращают тихую ошибку в понятное сообщение: «на листе „ЖД“ нет строки „Оборот вагона, сут“».

Что здесь справедливо. при каждом намеренном изменении структуры скрипт придётся обновлять. Поэтому стоит договориться с теми, кто ведёт исходники: структуру меняем заранее и сообщаем, а не «по ходу».

В разговоре с коллегой: *«Давай договоримся: структуру исходников меняем только с предупреждением, тогда скрипт будет работать без сюрпризов.»*

3. Шаблон вместо дизайна в коде

Частая ошибка — просить ИИ «нарисовать» презентацию в корпоративных цветах прямо в коде. Получается сотня строк про шрифты и отступы, а при смене брендбука всё переделывать. Надёжнее разделить **оформление** и **данные**:

- **Шаблон** — ваша ручная презентация прошлого месяца. Меняющиеся тексты заменены метками вроде {{период}} или {{выводы}}, таблицам и графикам даны имена, в ячейках таблиц стоят прочерки.
- **Скрипт** только заполняет шаблон: заменяет метки, пишет числа в ячейки, подставляет данные в графики. Цвета, шрифты, логотип он не трогает.



шаблон: метки {{…}} и прочерки *результат: сентябрь 2026*

Рис. 3. Шаблон + данные = презентация. Оформление живёт в шаблоне, цифры — в Excel

Пример

Имя таблице или графику в PowerPoint даётся так: «Главная → Выделить → Область выделения», двойной щелчок по названию объекта. В примере проекта таблицы называются «Таблица_выручка», «Таблица_Флот», графики — «График_динамика» и т. д. По этим именам скрипт их и находит.

4. План: собираем по частям

Просить «напиши весь скрипт по спецификации» — та же ошибка, что пять задач в одном промпте (урок 5). Модель что-то упустит, и искать, что именно, придётся во всём коде сразу. Работаем цепочкой: маленький шаг → запуск → проверка → следующий шаг.



Рис. 4. Шесть шагов проекта. После каждого — запуск и проверка

Шаг	Что проверяем после запуска
1. Прочитать и проверить исходники	Скрипт печатает прочитанные цифры; на «сломанном» файле — понятная ошибка
2. Посчитать показатели	2–3 показателя совпадают с расчётом в Excel
3. Заполнить один слайд	Слайд выглядит как в ручной версии, оформление не сбилось
4. Все таблицы и тексты	Не осталось ни одной метки {{…}} и ни одного прочерка
5. Графики	Ось и подписи на месте, число месяцев совпадает с исходником
6. Контрольный слайд	Контрольные суммы совпадают с итогами в Excel

По слайду, целиком или по шагам?

Частый вопрос: писать промпт на каждый слайд или один большой на всю презентацию? Ни то, ни другое. Дробим по **шагам работы**, а не по слайдам:

Стратегия	Что происходит	Итог
Один промпт на всю презентацию	Модель пишет весь скрипт сразу; часть правил из спецификации теряется, ошибку приходится искать во всём коде	Не рекомендуем
Отдельный промпт на каждый слайд	11 разговоров и 11 похожих кусков кода: чтение Excel, проверки и форматирование чисел повторяются и расходятся между собой	Не рекомендуем
По шагам (рис. 4)	Общие части пишутся один раз: чтение и проверка, расчёт. Один слайд делаем образцом, остальные — «по образцу» за 1–2 промпта по карте из спецификации	Рекомендуем

Каждый шаг дописывает **тот же самый файл**, а не создаёт новый. Поэтому в каждый промпт кладём спецификацию (spec.md) и текущую версию скрипта: «вот что уже есть, добавь шаг N». Если разговор стал длинным и модель начала путаться, откройте новый чат и начните с этих двух файлов — контекстное окно из урока 2 снова чистое.

5. Код: как просить и что получаем

Что делает скрипт целиком

Сначала посмотрим на итог всех шагов сверху. Скрипт один, и делает он четыре вещи подряд: читает и проверяет Excel, считает показатели, вносит данные в слайды шаблона и сохраняет готовый pptx. Так выглядит его главная часть — упрощённо и русскими словами вместо настоящих имён:

```
Сборка презентации за месяц:
  # Блок 1. Чтение и проверка исходников (шаг 1)
  данные = прочитать_и_проверить(папка_месяца)
  если есть ошибки: показать список ошибок и ОСТАНОВИТЬСЯ

  # Блок 2. Расчёт показателей (шаг 2)
  показатели = посчитать(данные)   # исполнение плана, рентабельность, Долг/EBITDA

  # Блок 3. Внесение данных в слайды (шаги 3–5)
  презентация = открыть("template.pptx")
  заполнить_таблицы(презентация, показатели)   # «—» в ячейках → числа
  заполнить_графики(презентация, данные)       # новые ряды в готовые графики
  заменить_метки(презентация)                  # {{период}}, {{выводы}}, комментарии

  # Блок 4. Сохранение и контроль (шаг 6)
  сохранить(презентация, "out/Отчёт_2026-09.pptx")
```

Ниже — промпты для всех шести шагов и два фрагмента готового скрипта. Полный файл — examples/monthly_pptx/build_presentation.py (около 300 строк с комментариями), а все промпты целиком, чтобы их было удобно копировать, — examples/monthly_pptx/prompts.md.

Как устроены промпты

Каждый шаг — отдельный промпт по шаблону из урока 4. Важная деталь для конфиденциальности: **модели не нужны реальные цифры**. Ей достаточно spec.md, структуры файлов и выдуманного примера. Реальные данные увидит только скрипт — на вашем компьютере или в локальной системе.

Промпты шагов 2–6 начинаются и заканчиваются одинаково, поэтому ниже показана только их середина:

```
Вот spec.md и текущая версия build_presentation.py.   ← начало
[задача шага]
Остальные части скрипта не меняй. Покажи файл целиком.  ← конец
```

Шаг 1. Чтение и проверка исходников

```
Я финансист, не программист. Пишем скрипт на Python по шагам
по приложенной спецификации spec.md. Сейчас шаг 1: чтение
и проверка исходников.

Пример листа «Сводка» (числа выдуманы):
Направление | План выручки, млн руб. | Факт выручки, млн руб. |
EBITDA, млн руб.
Флот | 500 | 512 | 159  …  Итого | 1620 | 1605 | 473

Что сделать: прочитать все листы из раздела 2 спецификации, строки
искать по подписи в первой колонке. Выполнить все проверки из
раздела 3. Если проверка не прошла — не падать с ошибкой Python,
а вывести понятный список проблем по-русски и остановиться.
Если всё в порядке — пока просто напечатать прочитанные цифры.
Библиотека: openpyxl. Добавь короткие комментарии по-русски.
```

Проверить: цифры в консоли совпадают с Excel; на копии файла с переименованной колонкой скрипт пишет понятную ошибку.

Фрагмент из блока 1 (проверка)

Читать код построчно не обязательно, но полезно узнать в нём свою спецификацию. Вот проверка «Итого» из готового скрипта:

```
total = sum(svodka[n][1] for n in DIRECTIONS)       # сумма направлений
if abs(total - svodka["Итого"][1]) > 0.5:            # не сходится с «Итого»?
    error(f"«Сводка»: сумма направлений {total} ≠ «Итого»")
```

Шаг 2. Расчёт показателей

```
Шаг 2: посчитай показатели по правилам из карты (раздел 4
спецификации): исполнение плана, рентабельность EBITDA,
Долг/EBITDA и статус ковенанта, изменения по направлениям.
Числа форматируй, как в разделе 4: «1 605», «29,5», минус «−».
Расчёт — отдельной функцией. Пока только напечатай результат,
презентацию не трогай.
```

Проверить: 2–3 показателя пересчитать в Excel вручную.

Шаг 3. Один слайд — образец

```
Шаг 3: заполни один слайд — № 6 «Флот». Шаблон template.pptx
приложен: таблица на слайде называется «Таблица_Флот», в её
ячейках стоят прочерки «—», под таблицей метка {{комментарий_Флот}}.
Замени прочерки числами, сохранив оформление ячеек; метку —
комментарием из kommentarii.xlsx дословно, с автором.
Сохрани результат в out/Отчёт_ГГГГ-ММ.pptx (месяц — из имени папки).
Цвета и шрифты в коде не задавай. Библиотека: python-pptx.
```

Проверить: слайд выглядит как в ручной версии, цифры на месте, оформление не сбилось.

Фрагмент из блока 3 (внесение данных на слайд)

А это заполнение таблицы на слайде: скрипт заменяет прочерк в ячейке на число, но не трогает оформление.

```
for i, row in enumerate(rows, start=1):          # строки данных (0-я — заголовок)
    for j, value in enumerate(row):              # ячейки строки
        cell = table.cell(i, j).text_frame.paragraphs[0]
        cell.runs[0].text = value                # вместо прочерка — число
```

Шаг 4. Остальные слайды «по образцу»

```
Шаг 4: слайд 6 «Флот» уже заполняется правильно. По его образцу
заполни таблицы слайдов 2, 5, 7, 8, 9 и все метки {{…}} по карте
из раздела 4 спецификации. Используй уже написанные функции
чтения, проверки и форматирования чисел, новых не дублируй.
```

Проверить: во всей презентации не осталось ни одной метки {{...}} и ни одного прочерка.

Шаг 5. Графики

```
Шаг 5: в шаблоне два графика. «График_план_факт» — столбцы «План»
и «Факт» по трём направлениям; «График_динамика» — линии по месяцам
для трёх направлений, число месяцев берётся из листа «Динамика»
и растёт каждый месяц. Подставь данные через замену данных
графика, чтобы оформление рядов из шаблона сохранилось.
```

Проверить: на графике динамики столько месяцев, сколько в Excel; цвета рядов — как в шаблоне.

Шаг 6. Служебный слайд и итог

```
Шаг 6: заполни метку {{контроль}} списком: какие файлы прочитаны,
контрольные суммы (план, факт и EBITDA итого), Долг/EBITDA,
напоминание «удалить слайд перед отправкой». После сохранения
выведи путь к файлу и контрольные суммы. Если в презентации
остались незаполненные метки {{…}} — выведи предупреждение.
```

Проверить: контрольные суммы на служебном слайде совпадают с итогами в Excel.

Главное

После шага 6 скрипт готов к приёмке (раздел 7). В этом уроке его пишут все шесть шагов. Дальше по курсу проект продолжится: в уроке 10 ИИ-агент будет сам вносить доработки в файлы проекта; в уроке 11 добавим автоматические тесты, которые сверяют цифры на слайдах с исходниками; в уроке 16 появятся черновик выводов от локальной модели и «кнопка» сборки в чате.

Осторожно

Если на каком-то шаге модель «заодно улучшила» уже работающую часть, это повод насторожиться. Попросите менять только то, что относится к текущему шагу, и после каждого шага сохраняйте рабочую версию файла. В уроке 10 для этого появится git — кнопка «отмена» для всего проекта.

6. Запуск: два пути

Скрипту нужен Python с двумя библиотеками: python-pptx (презентации) и openpyxl (Excel). Есть два пути:

Путь	Как	Кому подходит
На своём компьютере	Установить Python, затем в командной строке: pip install python-pptx openpyxl и python build_presentation.py data/2026-09	Тому, кто ведёт проект и будет его дорабатывать
В песочнице ИИ-ассистента	Загрузить в чат скрипт, шаблон и файлы месяца и попросить: «запусти скрипт без изменений и дай скачать результат»	Коллегам, которые не хотят ничего устанавливать

Во втором случае модель не пишет код, а только запускает готовый. Но она может «помочь» и поменять скрипт, если что-то пойдёт не так. Поэтому в просьбе явно сказано «без изменений», а первые запуски сверяются с ручной версией. Совсем надёжный вариант — сделать скрипт отдельным инструментом агента, который модель не может изменить. Это тема уроков 14 и 16.

Быстро устаревает — проверяйте актуальное

Песочницы для выполнения кода есть во многих ИИ-ассистентах, в том числе в локальных системах. Например, в LibreChat 0.8.7 есть Code Interpreter: файлы загружаются в чат, код выполняется в изолированной среде, результат можно скачать. Развёрнута ли такая песочница у вас и есть ли в ней библиотека для pptx, уточните у администратора: это отдельный сервис, который включают специально.

7. Исправление и приёмка

Сообщать об ошибке

Ошибки будут — это нормальная часть цикла (урок 8, рис. 3). Модели нужно то же, что и коллеге: **что делали, что ожидали, что получили**. Скопируйте сообщение об ошибке целиком — в нём обычно написано, где сломалось.

```
Запустил шаг 4 на данных за сентябрь. Ожидал: в таблице
«Флот» все ячейки заполнены. Получил: в колонке «Изменение»
для строки «Загрузка флота, %» стоит «+2,5», а в соседних
ячейках «86,5» и «89» — разное число знаков после запятой.
Нужно: одинаковое число знаков во всей строке. Поменяй
только форматирование этой таблицы, остальное не трогай.
```

Приёмка

Проект готов не тогда, когда «скрипт отработал», а когда пройдены три проверки из спецификации.

1. **Прошлый месяц.** Запускаем на данных прошлого месяца и сверяем с презентацией, которую тогда собрали вручную. Каждое расхождение разбираем: кто прав?
2. **«Сломанные» данные.** Нарочно переименовываем колонку и портим итог — скрипт должен остановиться с понятным сообщением.
3. **Два месяца в работе.** Презентацию собирает скрипт, человек проверяет по служебному слайду и выборочно. Только после этого ручную сборку отменяют.

Что сверяем (август)	Вручную	Скрипт	Итог
Факт выручки, итого	1 550	1 550	✓
EBITDA, итого	454	454	✓
Исполнение плана «Флот», %	103,3	103,1	? *вручную взяли предварительный факт 496 вместо 495 — прав скрипт*
Долг / EBITDA	3,16	3,16	✓
Комментарии подразделений	совпадают	совпадают	✓

Каждое расхождение разбирают: иногда ошибается скрипт, иногда — ручная версия

Рис. 5. Приёмка на прошлом месяце: расхождение нашлось — и ошибся не скрипт

```
Презентация НЕ собрана: исходники не прошли проверку.
  • Лист «ЖД»: нет строки «Оборот вагона, сут»
  • «Сводка»: факт выручки — сумма направлений 1 605 ≠ строке «Итого» 1 650
Исправьте исходники (или обновите скрипт, если структура изменилась
намеренно) и запустите снова.
```

Так выглядит проверка 2 в примере проекта: скрипт нашёл обе подложенные ошибки и ничего не собрал.

Текстовые выводы

В презентации два вида текста, и обращаться с ними нужно по-разному:

- **Комментарии подразделений** скрипт вставляет **дословно** из файла комментариев. ИИ их не трогает: за формулировки отвечает автор.
- **Выводы, которые сейчас пишут сами**, — кандидат на черновик от ИИ по уже посчитанным цифрам. Это отдельный шаг с проверкой человеком, и делать его мы будем в уроке 16. Пока на слайде «Выводы» — текст из файла комментариев.

Частое сомнение

«Выводы для руководства нельзя доверять ИИ»

Согласны — нельзя доверять без проверки. Поэтому в проекте выводы либо пишут люди (и скрипт вставляет их дословно), либо ИИ готовит черновик по уже посчитанным цифрам, а человек его правит и подписывает. Цифры в выводах при этом берутся из тех же расчётов, что и на слайдах.

Что здесь справедливо. даже хороший черновик может звучать убедительно и при этом расставлять акценты не так, как сказал бы автор. Решение, что отправлять руководству, всегда за человеком (урок 7, лестница проверки: «эксперт и подпись»).

В разговоре с коллегой: *«Пусть ИИ готовит черновик выводов, а подписываем их мы.»*

Глоссарий урока

Термин	Простыми словами
Спецификация	Точное описание задачи: что делаем, откуда данные, как считаем, как проверяем
Карта «слайд → источник → правило»	Главная часть спецификации: для каждого слайда — откуда цифры и как их считать
Проверка входа	Проверка исходников до сборки: файлы, листы, заголовки, итоги
Шаблон	Презентация-образец с метками {{…}} и именованными таблицами и графиками
Метка	Текст вида {{период}}, который скрипт заменяет на значение
Библиотека python-pptx	Готовый инструмент Python для чтения и заполнения презентаций
Песочница выполнения кода	Изолированная среда в ИИ-ассистенте, где код запускается без установки на ваш компьютер
Приёмка	Проверка, что проект делает то, что нужно: прошлый месяц, «сломанные» данные, два месяца в работе
Служебный слайд	Слайд с контрольными суммами для проверяющего; удаляется перед отправкой

Вопросы для самопроверки

1. На какие три вопроса нужно ответить до начала проекта?
2. Кто должен писать спецификацию и почему?
3. Почему строки в исходниках лучше искать по подписи, а не по номеру?
4. Зачем отделять шаблон от скрипта, а не задавать цвета и шрифты в коде?
5. Почему скрипт собирают по шагам, а не просят написать целиком?
6. Как сообщить модели об ошибке, чтобы она её исправила с первого раза?
7. Из каких трёх проверок состоит приёмка проекта?

Ответы для проверки

Ответы

1. Что автоматизируем, что не автоматизируем и как поймём, что получилось.
2. Владелец отчёта: только он знает, откуда каждая цифра, как её считать и округлять.
3. Номер строки сдвинется, как только кто-то вставит строку; подпись «Итого» останется прежней.
4. Оформление остаётся у шаблона и дизайнера: сменился брендбук — меняем шаблон, а не код. И кода становится в разы меньше.
5. Маленький шаг легко проверить и исправить; в большом скрипте ошибку приходится искать везде сразу.
6. Что делали, что ожидали, что получили, плюс текст ошибки целиком; и попросить менять только нужное.
7. Прогон на прошлом месяце со сверкой с ручной версией; прогон на «сломанных» данных; два месяца в работе с проверкой человеком.

Практика

Практика

Задание 1. Спецификация своего отчёта (20 мин)

Возьмите регулярный отчёт или презентацию и составьте спецификацию с моделью по промпту шага 0 (раздел 2), сверяясь с образцом spec.md из примера: цель и границы, входные данные, проверки входа, карта «слайд → источник → правило» хотя бы для трёх слайдов.

Задание 2. Первый шаг (15 мин)

Попросите модель написать шаг 1 — чтение и проверку исходников — на выдуманном примере ваших данных. Запустите на настоящем файле, затем на копии с переименованной колонкой. Понятна ли ошибка?

Практика

Задание 3. Пример проекта (10 мин)

Откройте папку examples/monthly_pptx, запустите скрипт на сентябре и на августе (или попросите ассистента запустить его в песочнице). Измените одну цифру в «Сводке» так, чтобы «Итого» не сошлось, и посмотрите, что скажет скрипт.

Что почитать

- README и spec.md в папке examples/monthly_pptx — готовая основа для собственного проекта.
- Документация python-pptx (python-pptx.readthedocs.io), разделы о таблицах и графиках.
- Следующий урок: **«Агентный режим кодинга»** — ИИ сам правит файлы проекта, правила проекта в файле-инструкции и git как кнопка «отмена».